

RAPPORT DE PROJET FINAL

Infrastructure Data Cloud et pipeline ETL



Titre du projet :

**Pipeline Data Cloud -
Assemblée nationale
française, 17^e législature**



Auteur :

Samir NZAMBA



Formation :

**Mastère DPIA 1 - Directeur de
Projet en Intelligence Artificielle**



Établissement :

Fonderie de l'image



Année académique :

2025-2026



Encadrant :

M. Antoine LUCSKO



Date de rendu :

26 mai 2026



Dépôt GitHub :

<https://github.com/SNZAMBA65/data-pipeline-emploi>



1) Introduction et contexte

Les données publiques françaises sont disponibles depuis longtemps, mais leur exploitation reste souvent complexe. Dans le cas de l'Assemblée nationale, les données sont accessibles librement, cependant elles sont généralement proposées sous forme de fichiers volumineux, souvent en JSON imbriqué, parfois en XML, avec une structure peu homogène et une documentation insuffisante pour une utilisation directe.

Le problème principal n'est donc pas l'accès aux données, mais leur exploitation. En l'état, il faut souvent des compétences techniques avancées pour les nettoyer, les structurer et les croiser. Cela limite leur usage à des profils spécialisés, alors qu'elles pourraient être utiles dans un cadre beaucoup plus large, notamment académique.

Ce projet part de ce constat. L'objectif est de construire une infrastructure capable de transformer ces données brutes en informations structurées, propres, cohérentes et faciles à interroger. Le système automatise la collecte, le nettoyage et l'enrichissement des données issues de sources publiques. Ensuite, ces données sont stockées dans une architecture adaptée à l'analyse, combinant data lake et base relationnelle, puis exposées via un dashboard et des outils d'exploration.

L'idée n'est pas d'interpréter les données mais de les rendre exploitables de manière fiable, reproductible et simple à utiliser.

Dans ce projet, les données utilisées proviennent de l'Assemblée nationale. Elles sont exploitées dans un cadre académique, notamment pour des travaux en sciences sociales et sciences politiques.

Objectifs du projet

Le projet s'articule autour de quatre objectifs principaux. D'abord automatiser la collecte quotidienne des données parlementaires. Ensuite construire une infrastructure capable de les nettoyer et de les structurer correctement. Puis mettre en place un dashboard interactif pour permettre leur exploration sans compétences techniques. Enfin produire une analyse statistique de la 17e législature afin d'en faire ressortir les grandes tendances.

Périmètre

Le projet couvre les 577 députés actifs, les 6899 scrutins publics depuis juillet 2024 et les 12 groupes politiques de l'hémicycle. En revanche, il ne traite pas les votes nominatifs par député, ni les débats, ni les questions parlementaires, ni les données des législatures précédentes.

2) Architecture et infrastructure cloud

Vue d'ensemble

L'architecture repose sur une logique classique de pipeline de données. On retrouve plusieurs étapes distinctes : collecte, stockage brut, transformation, stockage structuré, puis restitution. Chaque étape est isolée dans un service dédié afin de garantir la clarté et la maintenabilité du système.

Architecture du pipeline ETL

Flux de données de la collecte à la visualisation

Extract	→	Data Lake (MinIO)	→	Transform	→	Data Warehouse (PostgreSQL)	→	Visualisation
API officielle AN ZIP AMO30 — tous acteurs depuis la XIe législature ZIP JSON scrutins Scraping HTML Groupes politiques		Données brutes JSON deputes/raw_YYYY-MM-DD.json scrutins/raw_YYYY-MM-DD.json groupes/raw_YYYY-MM-DD.json ⚠ 3,114 acteurs bruts ⚠ Valeurs XML brutes ⚠ Sans calcul d'âge ⚠ Sans mapping groupes ⚠ Toutes législatures mélangées		DataCleaner etl/transform/ cleaner.py ✓ Filtre 17e législature 2,537 ignorés ✓ Nettoyage @xsi:nil ✓ Calcul de l'âge ✓ Déduction du genre ✓ Mapping groupe→sigle		Tables normalisées deputes - scrutins groupes_politiques pipeline_stats ✓ 577 députés actifs ✓ 6,673 scrutins ✓ 12 groupes mappés ✓ Données normalisées		Streamlit Dashboard Ce dashboard Notebooks Jupyter Analyse en 4 actes Grafana Monitoring infra

Architecture pipeline ETL

Services utilisés

Le projet s'appuie sur plusieurs services.

Docker constitue l'environnement principal d'exécution du système. L'ensemble de l'infrastructure est entièrement conteneurisé via Docker Compose, ce qui permet un déploiement reproductible, portable et facilement maintenable.

MinIO sert de data lake pour stocker les données brutes au format JSON.

PostgreSQL 15 joue le rôle de data warehouse pour les données structurées.

Apache Airflow 2.9 gère l'orchestration du pipeline.

Prometheus collecte les métriques système.

Grafana permet leur visualisation.

Streamlit sert à construire le dashboard analytique.

Le projet a également été testé sur une instance AWS EC2 afin de valider sa portabilité et son fonctionnement dans un environnement distant, mais l'infrastructure principale repose entièrement sur Docker et Docker Compose.

Data lake et data warehouse

MinIO stocke les données brutes sans transformation. Chaque exécution du pipeline génère des fichiers JSON horodatés, organisés par type de données.

PostgreSQL contient les données nettoyées et structurées. On y retrouve principalement les députés, les scrutins, les groupes politiques et les statistiques du pipeline.

Sécurité

Les accès sont sécurisés via des variables d'environnement stockées dans un fichier .env exclu du dépôt Git grâce au .gitignore. Les mots de passe ne sont jamais versionnés.

3) Docker et reproductibilité

Docker joue un rôle central dans le projet. Il permet de reproduire l'environnement complet en une seule commande, que ce soit en local ou sur le serveur.

Tous les services sont définis dans un fichier docker-compose.yml unique. On y retrouve MinIO, PostgreSQL, Airflow, Prometheus, Grafana et pgAdmin.

L'infrastructure peut ensuite être démarrée avec la commande docker compose up.

Chaque service est configuré avec des mécanismes de redémarrage automatique et des contrôles de santé afin de garantir la stabilité globale du système. L'ensemble est pensé pour fonctionner de manière autonome une fois lancé.

```
samir@DESKTOP-PVEMP35 MINGW64 ~/Desktop/data-pipeline-emploi (main)
$ docker compose ps
NAME                                IMAGE                                COMMAND                                SERVICE                                CREATED
STATUS                                PORTS
airflow_scheduler                   apache/airflow:2.9.1               "/usr/bin/dumb-init ..."          airflow_scheduler                    3 minutes
go Up 3 minutes                    8080/tcp
airflow_webserver                   apache/airflow:2.9.1               "/usr/bin/dumb-init ..."          airflow_webserver                    3 minutes
go Up 3 minutes (healthy)          0.0.0.0:8080->8080/tcp
grafana                             grafana/grafana:latest             "/run.sh"                             grafana                              3 minutes
go Up 3 minutes                    0.0.0.0:3000->3000/tcp
minio                               minio/minio:latest                 "/usr/bin/docker-ent..."          minio                                3 minutes
go Up 3 minutes (healthy)          0.0.0.0:9000-9001->9000-9001/tcp
pgadmin                             dpape/pgadmin4:latest              "/entrypoint.sh"                     pgadmin                             3 minutes
go Up 3 minutes                    443/tcp, 0.0.0.0:5050->80/tcp
postgres_airflow                   postgres:15-alpine                 "docker-entrypoint.s..."          postgres_airflow                    3 minutes
go Up 3 minutes (healthy)          5432/tcp
postgres_exporter                   prometheuscommunity/postgres-exporter:latest "/bin/postgres_expor..."          postgres-exporter                    3 minutes
go Up 3 minutes                    0.0.0.0:9187->9187/tcp
postgres_jobs                       postgres:15-alpine                 "docker-entrypoint.s..."          postgres                             3 minutes
go Up 3 minutes (healthy)          0.0.0.0:5432->5432/tcp
prometheus                         prom/prometheus:latest             "/bin/prometheus --c..."          prometheus                          3 minutes
go Up 3 minutes                    0.0.0.0:9090->9090/tcp
```

Docker compose ps montrant les 9 services Up.

```
# Infrastructure locale du pipeline de données.
# Chaque service est isolé dans son conteneur Docker pour faciliter le déploiement,
# l'observation des métriques et le redémarrage propre de l'environnement.
▷ Run All Services
services:

# MinIO : data lake objet utilisé pour stocker les données brutes JSON.
▷ Run Service
minio:
  image: minio/minio:latest
  container_name: minio
  ports:
    - "9000:9000" # API S3 du data lake
    - "9001:9001" # Console web MinIO
  environment:
    MINIO_ROOT_USER: ${MINIO_ROOT_USER}
    MINIO_ROOT_PASSWORD: ${MINIO_ROOT_PASSWORD}
    MINIO_PROMETHEUS_AUTH_TYPE: "public"
  volumes:
    - minio_data:/data
  command: server /data --console-address ":9001"
  healthcheck:
    test: ["CMD", "curl", "-f", "http://localhost:9000/minio/health/live"]
    interval: 30s
    timeout: 10s
    retries: 3
  restart: unless-stopped
```

Extrait commenté du fichier docker-compose.yml.

4) Pipeline ETL

Sources de données

Le pipeline repose sur deux sources principales.

La première est l'API officielle de l'Assemblée nationale. Elle fournit des fichiers ZIP contenant les données des acteurs parlementaires et des scrutins.

La deuxième source est un scraping HTML des groupes politiques, car ces informations ne sont pas disponibles sous une forme directement exploitable dans l'API.

Collecte

Le pipeline est exécuté automatiquement chaque jour à 6 heures via Airflow. Il peut aussi être lancé manuellement avec différents paramètres pour les tests.

Les erreurs sont gérées à deux niveaux. Les erreurs réseau sont capturées sans bloquer le processus, et Airflow gère des tentatives de relance automatiques.

Transformation

La phase de transformation est la plus importante du pipeline.

Le fichier AMO30 contient exactement 3114 acteurs couvrant plusieurs législatures. Après filtrage sur les mandats actifs de la 17e législature, seules 577 entrées sont conservées.

Ensuite, les données XML sont nettoyées, notamment les valeurs nulles mal formées.

Les dates sont converties dans un format standard, avec gestion des erreurs.

L'âge est recalculé à partir de la date de naissance afin de garantir la cohérence des données.

Le genre est déduit à partir de la civilité.

Enfin, les députés sont rattachés à leur groupe politique via un croisement entre les organes parlementaires et les mandats actifs.

```
samir@DESKTOP-PVEMP35 MINGW64 ~/Desktop/data-pipeline-emploi (main)
$ python etl/pipeline.py
2026-05-26 10:55:35.257 INFO cleaner:__init__:72 - Initialisation de DataCleaner
2026-05-26 10:55:35.257 INFO cleaner:transformer:95 - Début de la transformation des données
2026-05-26 10:55:35.270 INFO cleaner:_nettoyer_deputes:170 - Députés nettoyés : 577 actifs (0 sans groupe ignorés)
2026-05-26 10:55:35.504 INFO cleaner:_nettoyer_scrutins:247 - Scrutins nettoyés : 6899
2026-05-26 10:55:35.504 SUCCESS cleaner:transformer:103 - Transformation terminée - 577 députés / 6899 scrutins
2026-05-26 10:55:35.504 INFO __main__:run_pipeline:169 - ÉTAPE 4 : CHARGEMENT CLEAN → POSTGRESQL (Data Warehouse) —
2026-05-26 10:55:35.504 INFO postgres_loader:__init__:27 - Initialisation de PostgresLoader
2026-05-26 10:55:35.605 SUCCESS postgres_loader:connecter:46 - Connexion PostgreSQL établie
2026-05-26 10:55:35.610 SUCCESS postgres_loader:creer_tables:147 - Tables créées / vérifiées
2026-05-26 10:55:35.626 SUCCESS postgres_loader:insérer_groupes:194 - 12 groupes insérés/mis à jour
2026-05-26 10:55:35.644 INFO postgres_loader:insérer_deputes:210 - Table députés vidée avant rechargement
2026-05-26 10:55:36.356 SUCCESS postgres_loader:insérer_deputes:249 - 577 députés insérés/mis à jour
2026-05-26 10:55:36.504 SUCCESS postgres_loader:insérer_scrutins:311 - 6899 scrutins insérés/mis à jour
2026-05-26 10:55:46.232 SUCCESS postgres_loader:insérer_stats:343 - Statistiques pipeline insérées
2026-05-26 10:55:46.232 INFO __main__:run_pipeline:199 - =====
2026-05-26 10:55:46.232 SUCCESS __main__:run_pipeline:200 - PIPELINE TERMINÉ AVEC SUCCÈS
2026-05-26 10:55:46.233 INFO __main__:run_pipeline:201 - Groupes politiques : 12
2026-05-26 10:55:46.233 INFO __main__:run_pipeline:202 - — NETTOYAGE DÉPUTÉS —
2026-05-26 10:55:46.233 INFO __main__:run_pipeline:203 - Acteurs bruts (AM030) : 3,114
2026-05-26 10:55:46.233 INFO __main__:run_pipeline:207 - Ignorés (autres législatures) : 2,537
2026-05-26 10:55:46.234 INFO __main__:run_pipeline:211 - Retenus (17e lég. actifs) : 577
2026-05-26 10:55:46.234 INFO __main__:run_pipeline:212 - Députés nettoyés : 577
2026-05-26 10:55:46.234 INFO __main__:run_pipeline:213 - — SCRUTINS —
2026-05-26 10:55:46.234 INFO __main__:run_pipeline:214 - Scrutins collectés : 6,899
2026-05-26 10:55:46.235 INFO __main__:run_pipeline:215 - Scrutins nettoyés : 6,899
2026-05-26 10:55:46.235 INFO __main__:run_pipeline:216 - =====
```

Logs du pipeline montrant 3114 acteurs analysés et 577 députés retenus.

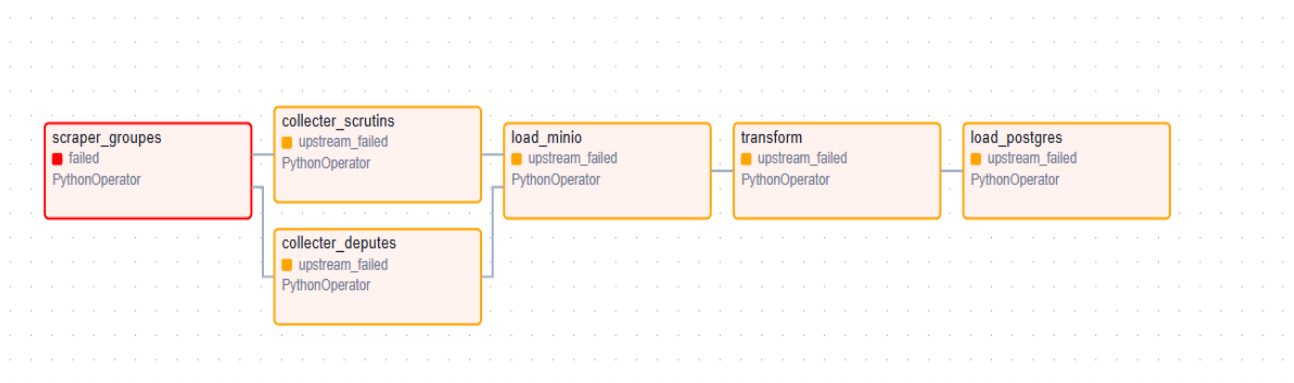
Chargement

Les données brutes sont envoyées vers MinIO. Les données transformées sont ensuite insérées dans PostgreSQL avec des mécanismes de type INSERT ON CONFLICT DO UPDATE afin d'éviter les doublons.

La table deputies est entièrement régénérée à chaque exécution afin de garantir la cohérence des données.

Orchestration

Le DAG Airflow organise six étapes principales, allant de la collecte jusqu'au chargement final dans la base de données. Les tâches sont reliées entre elles et les données transitent via XCom.



Interface Airflow avec le graphe du DAG pipeline_assemblee_nationale.

Difficultés rencontrées

La principale difficulté a concerné le mapping des groupes politiques. La structure des données de l'Assemblée nationale ne distingue pas clairement les groupes parlementaires des autres organes. Il a fallu croiser plusieurs niveaux d'information pour obtenir un rattachement fiable des députés à leur groupe politique.

Une autre difficulté importante a concerné la gestion des données résiduelles dans PostgreSQL. Lors des premiers tests, la table `deputes` contenait 596 lignes au lieu des 577 attendues. Le problème provenait d'anciens enregistrements conservés après plusieurs exécutions du pipeline.

L'utilisation d'un simple mécanisme d'upsert ne suffisait pas à garantir la cohérence lorsque la composition de l'hémicycle évoluait. La solution retenue a été de vider complètement la table via un `TRUNCATE` avant chaque rechargement intégral des données.

5) Analyse des données

Méthodologie

L'analyse s'organise autour d'une question simple : comment la fragmentation politique de la 17^e législature se reflète dans les données ?

Trois notebooks ont été utilisés pour explorer différents aspects : composition de l'hémicycle, analyse des scrutins et démographie des groupes politiques.

Les méthodes statistiques incluent des statistiques descriptives, des tests ANOVA, des tests non paramétriques, des tests du chi deux, ainsi que des mesures de fragmentation comme l'indice HHI.

Acte 1 : portrait de l'hémicycle

Les premières visualisations montrent la structure globale de l'Assemblée. On observe une fragmentation importante avec 12 groupes politiques et aucun bloc dominant.

L'indice HHI calculé sur la répartition des sièges est faible, ce qui confirme un niveau élevé de fragmentation parlementaire. Le groupe le plus important ne dépasse pas 21 % des sièges, rendant impossible toute majorité stable sans coalition.

Herfindahl-Hirschman Index

Notebook 01 Audit qualité et nettoyage des données parlementaires

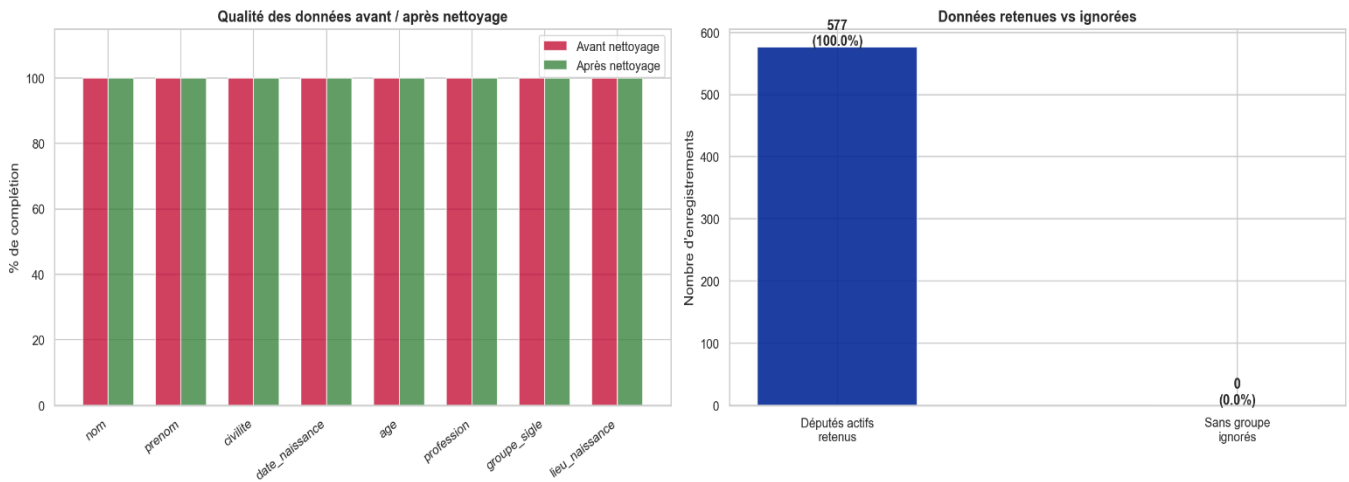


Figure 01_qualite_nettoyage.png

Acte 1 Portrait de l'hémicycle : une Assemblée fragmentée

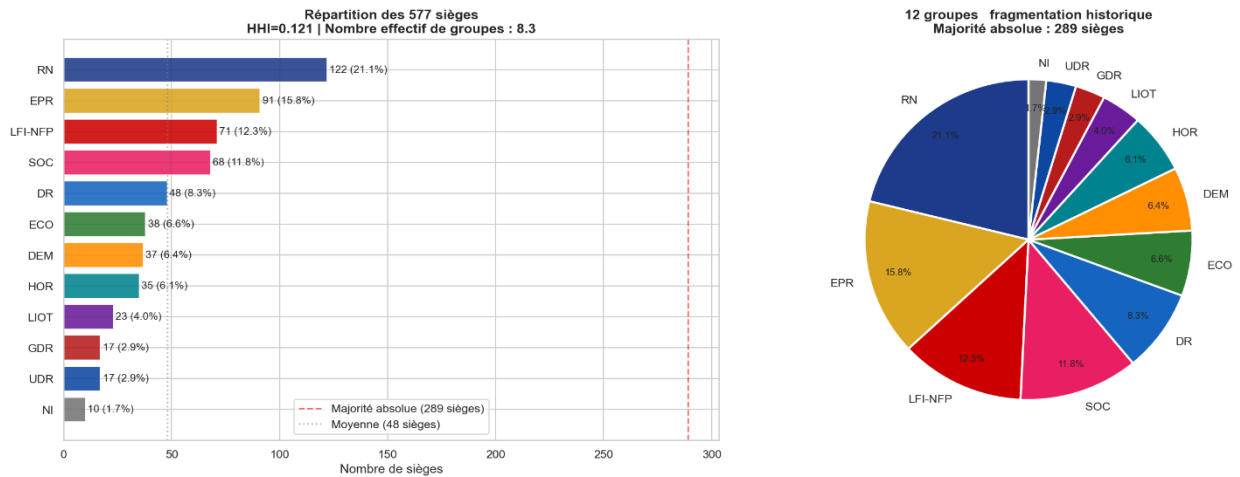


Figure 02_fragmentation_hemicycle.png

La population parlementaire est composée à 37 % de femmes, avec un âge moyen de 51 ans. Les tests statistiques montrent que les différences d'âge et de parité entre groupes sont significatives.

Acte 1 Portrait démographique des 577 députés actifs

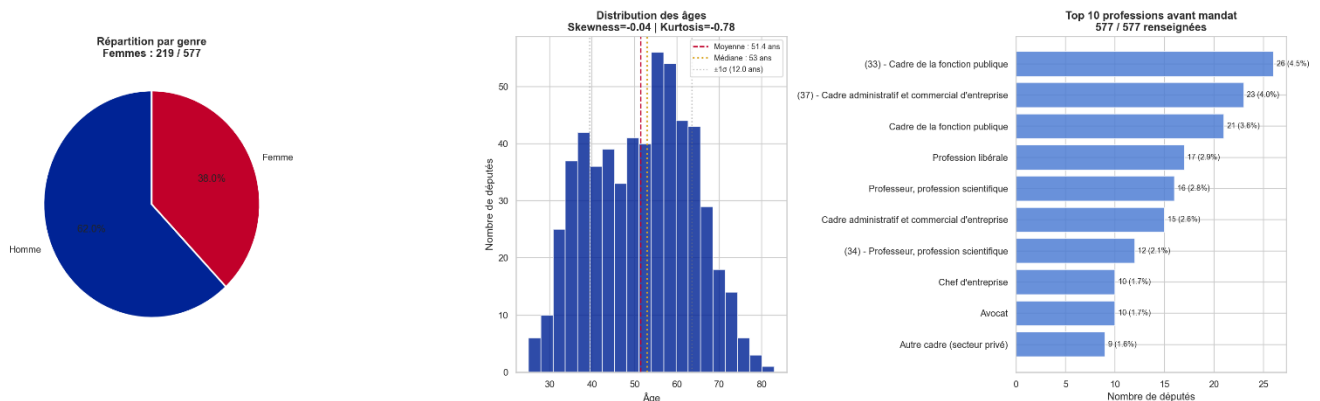


Figure 03_portrait_demographique.png

Acte 1 Les groupes ont-ils des profils démographiques distincts ?

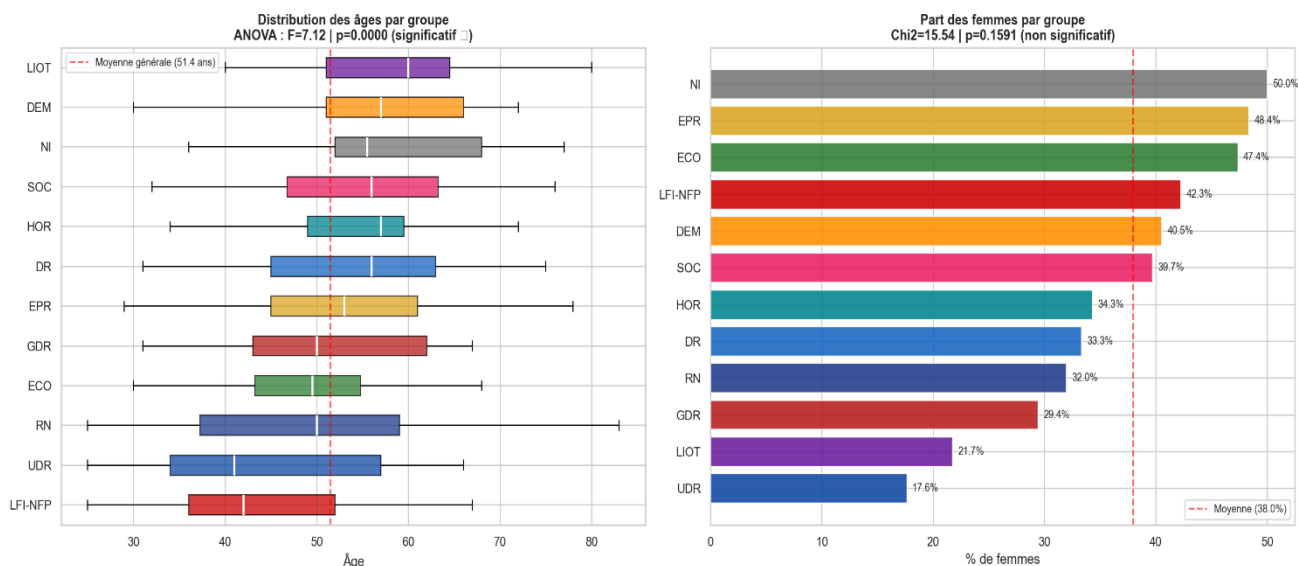


Figure 04_demographie_par_groupe.png

Actes 2 et 3 : une Assemblée qui dit non

Le taux de rejet des textes atteint 64,7 %, ce qui est particulièrement élevé. Dans la majorité des cas, les textes ne sont pas adoptés.

L'activité parlementaire est concentrée sur certaines périodes, notamment les périodes budgétaires.

La participation moyenne reste relativement faible, autour de 27 %.

Acte 2 Une Assemblée qui dit non : taux de rejet historique

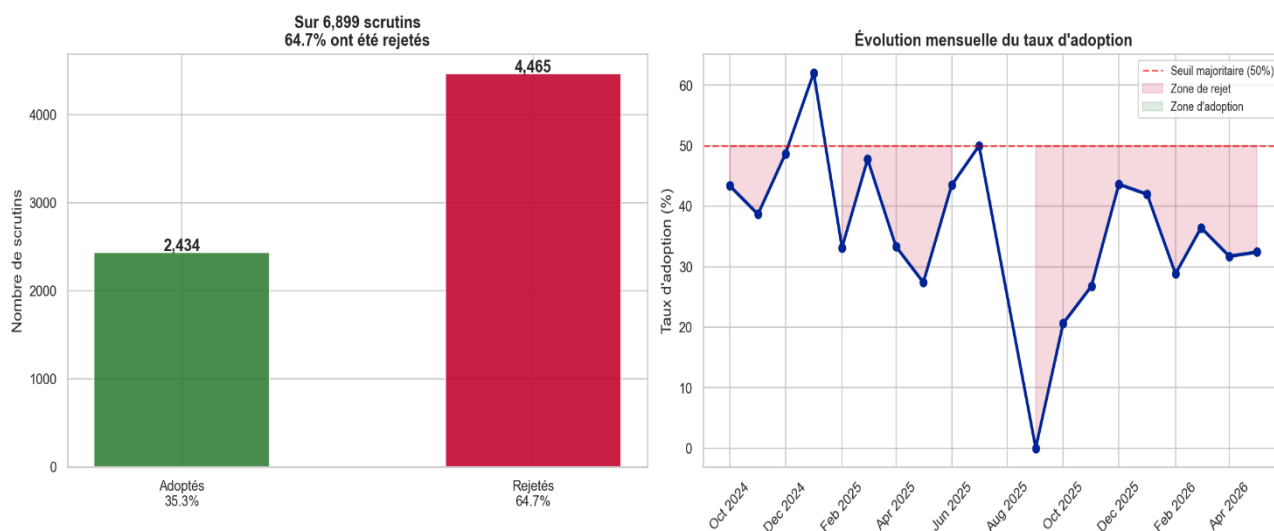


Figure 04_taux_rejet.png

Acte 2 Rythme et intensité de l'activité parlementaire

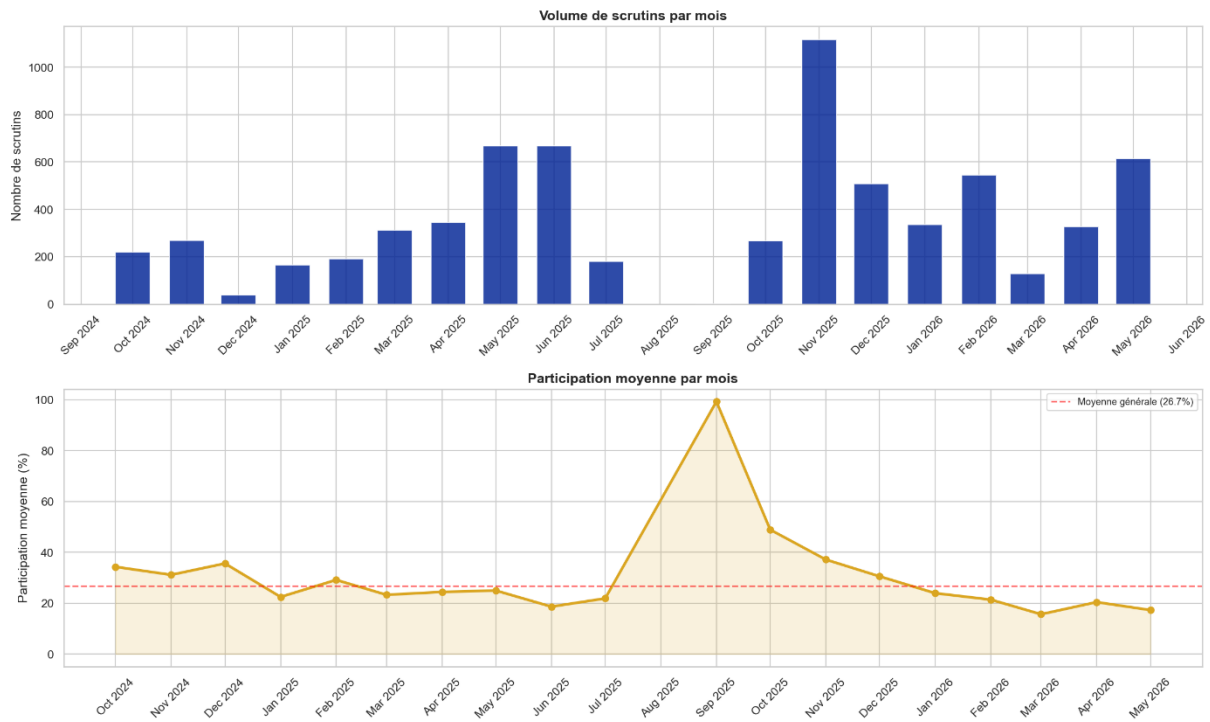


Figure 05_activite_participation.png

Acte 3 L'anatomie des votes serrés : quand chaque voix compte

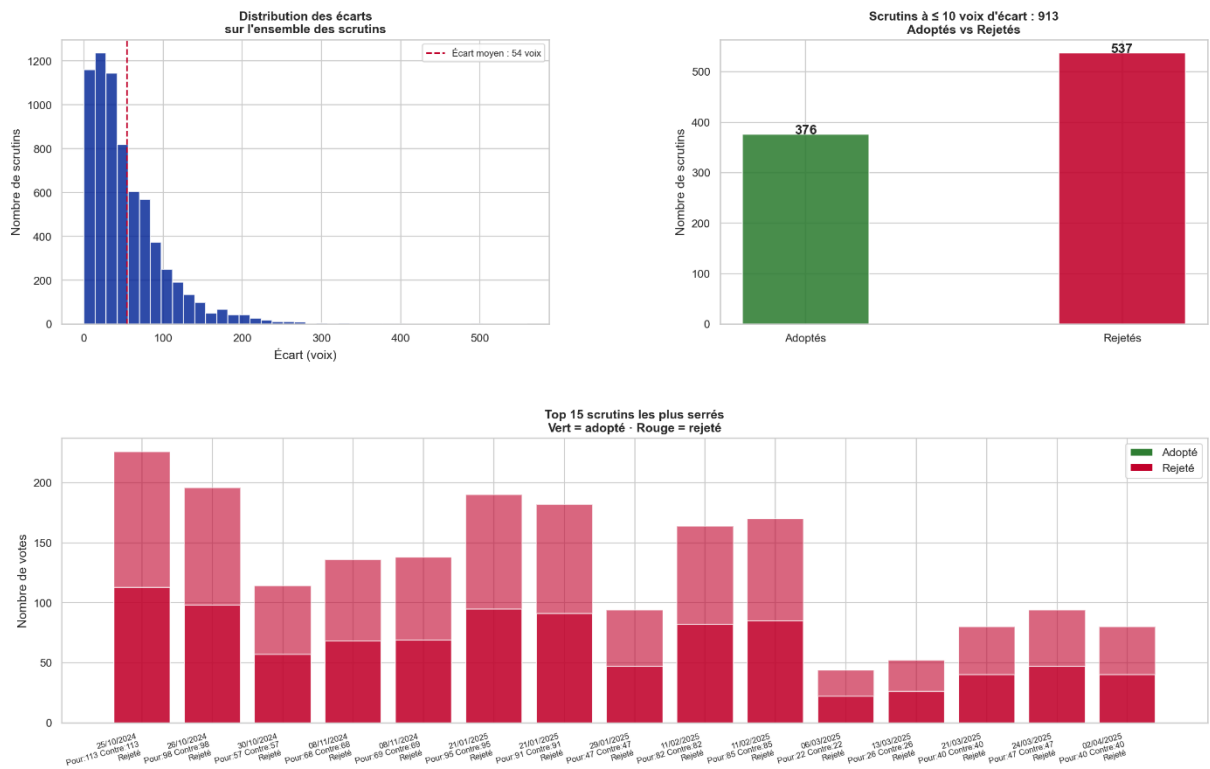


Figure 07_scrutins_serres.png

Les scrutins les plus serrés montrent parfois des écarts extrêmement faibles, voire des égalités parfaites entre votes Pour et Contre.

Acte 4 : démographie du pouvoir

Les groupes politiques présentent des profils sociologiques différents. Les tests statistiques confirment que ces différences ne sont pas dues au hasard.

Acte 4 Les professions avant mandat révèlent-elles une sociologie politique ?

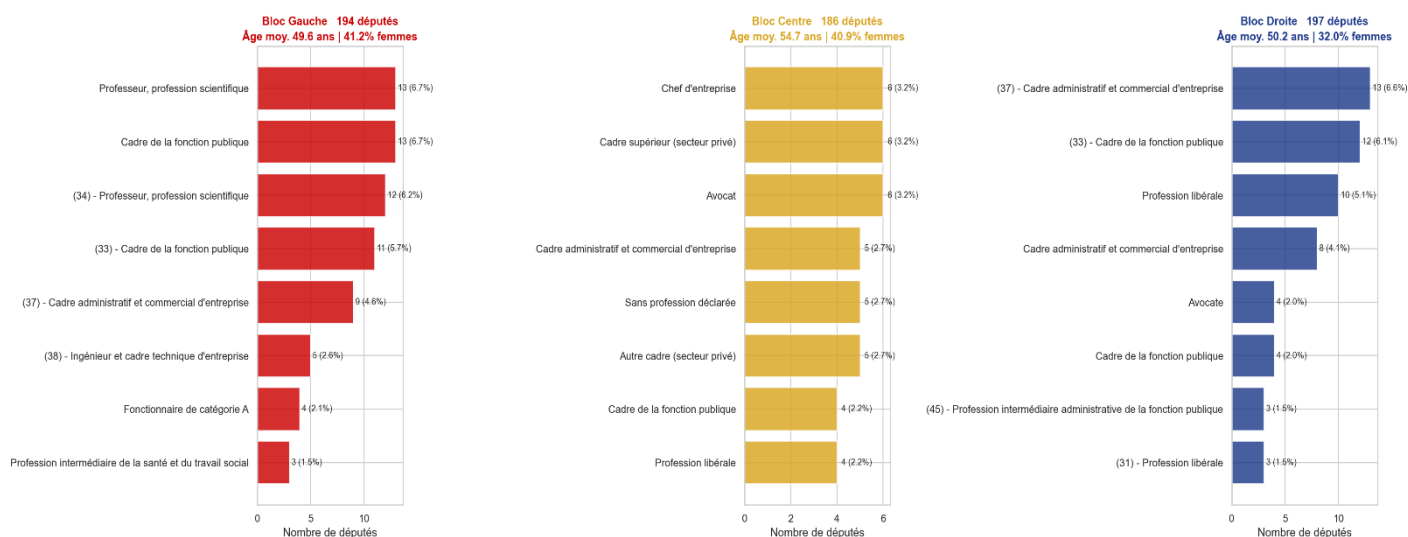


Figure 10_professions_par_bloc.png

Acte 4 Démographie par groupe : âge et parité

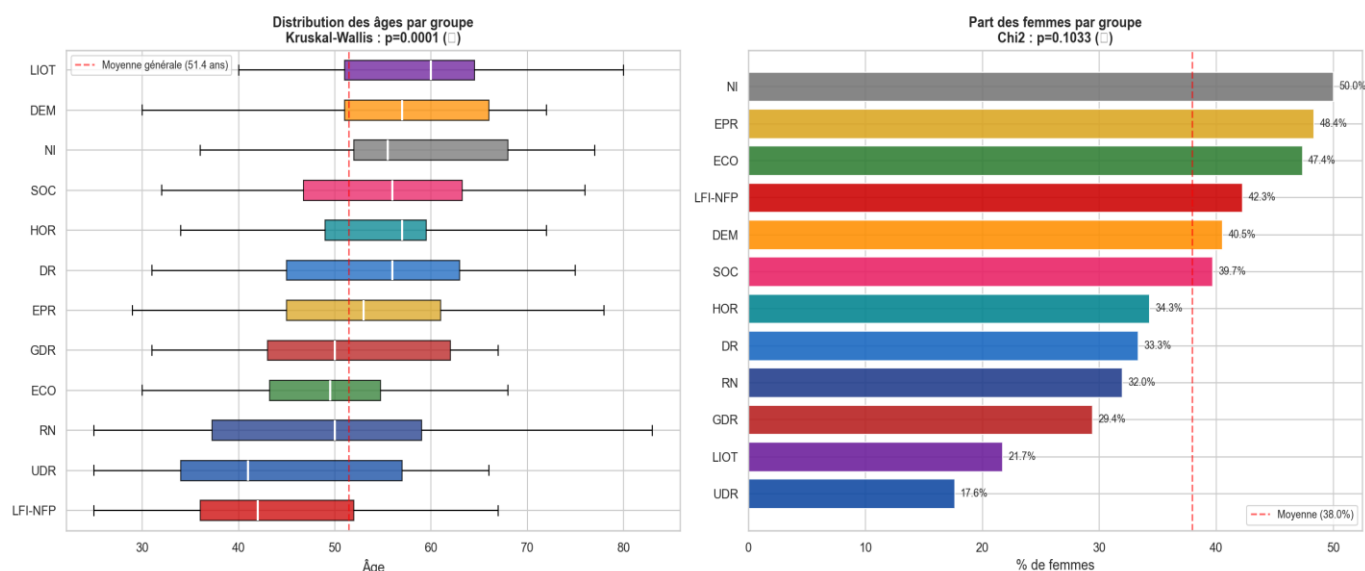


Figure 11_age_genre_par_groupe.png

Insights

Trois conclusions principales ressortent de l'analyse.

D'abord, la fragmentation de l'Assemblée est réelle et mesurable dans les données.

Ensuite, cette fragmentation explique en grande partie le taux élevé de rejet des textes, chaque vote nécessitant des compromis ou des alliances temporaires.

Enfin, les différences entre groupes ne sont pas uniquement politiques mais aussi sociologiques.

6) Dashboard interactif

Technologies

Le dashboard est développé avec Streamlit et Plotly.

Les données sont récupérées depuis PostgreSQL via SQLAlchemy et mises en cache grâce aux mécanismes `st.cache_data` et `st.cache_resource` de Streamlit afin d'améliorer les performances et de limiter les requêtes répétitives vers la base de données.

Structure

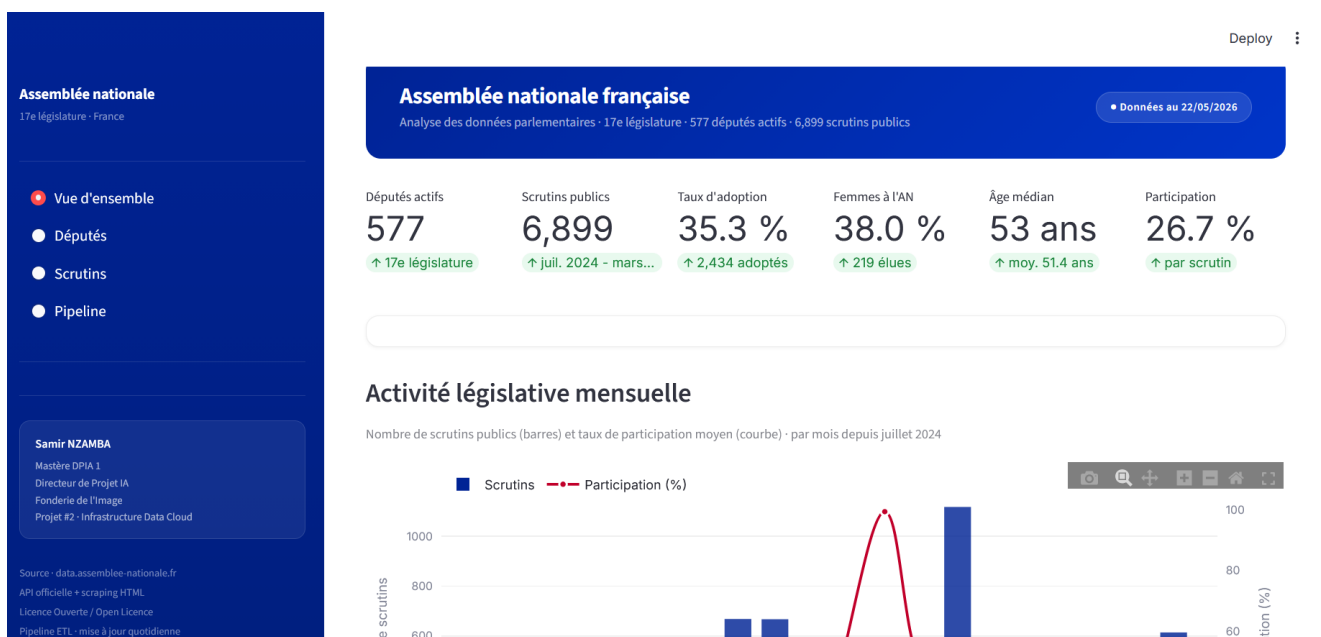
Le dashboard est organisé en quatre sections.

Une page de synthèse avec les indicateurs principaux.

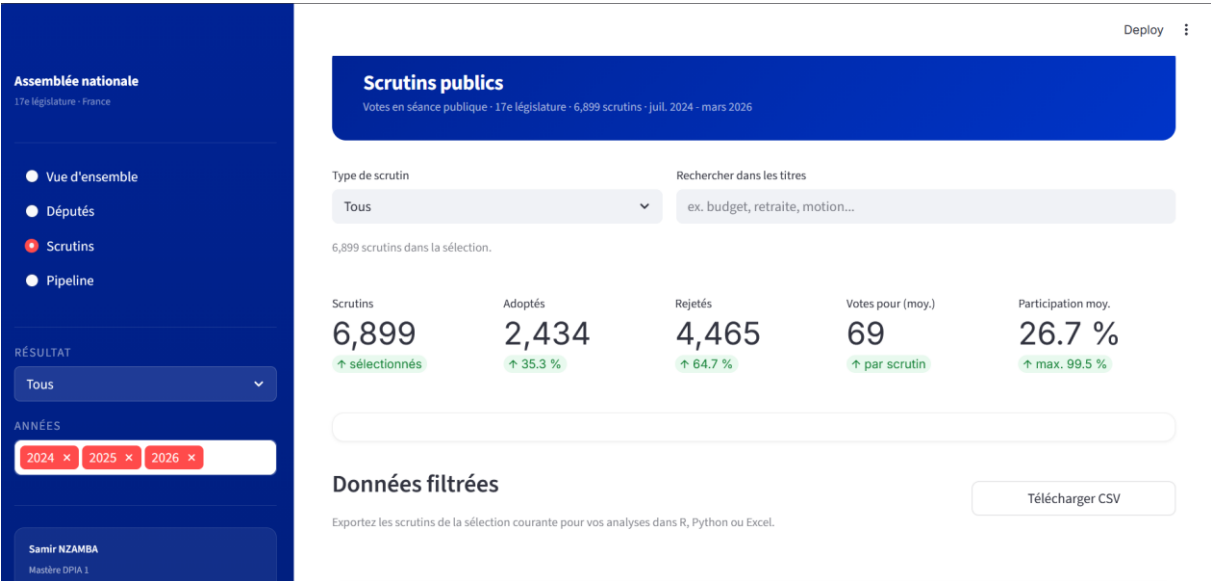
Une page dédiée aux députés avec filtres et visualisations.

Une page consacrée aux scrutins avec recherche et export de données.

Une page technique qui documente le pipeline.



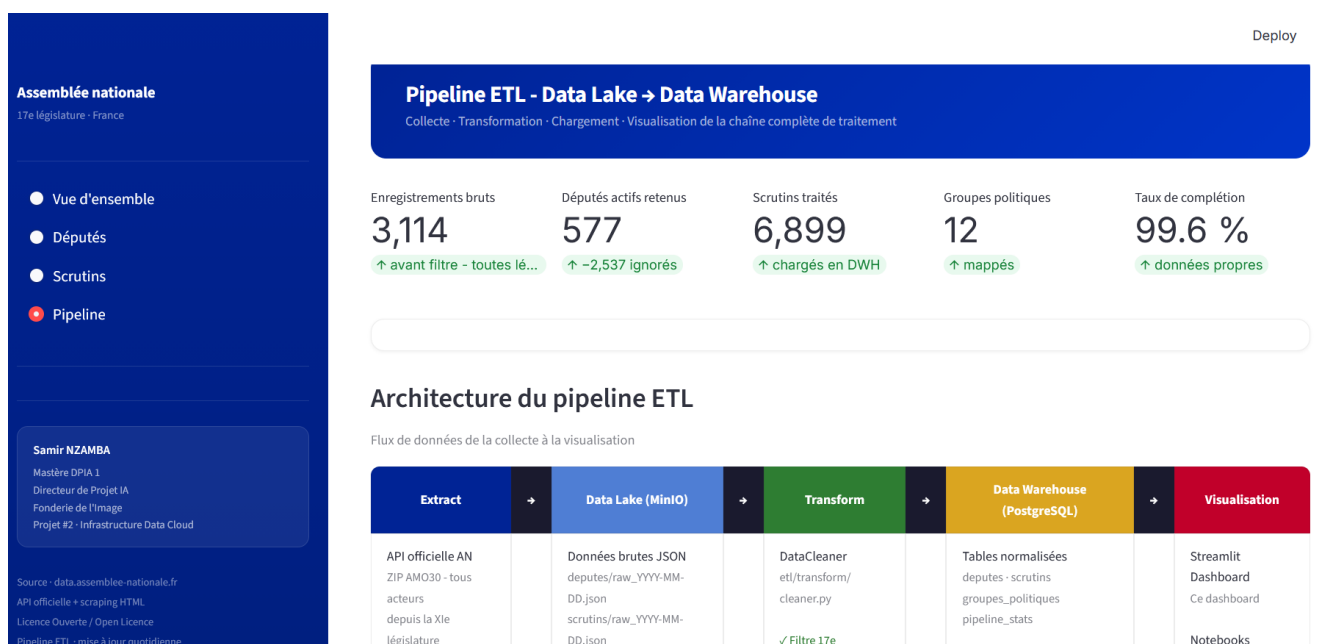
Page d'accueil du dashboard.



Page Scrutins avec recherche active.



Page Députés avec filtres actifs.



Page Pipeline.

Limites

Le dashboard ne permet pas encore d’analyser les votes individuels, car ces données ne sont pas incluses dans le périmètre du projet.

Il n’est pas non plus accessible en continu sur une infrastructure publique permanente.

7) Gestion de projet et GitHub

Organisation

Le projet a été construit en trois grandes phases. D’abord l’infrastructure, ensuite le pipeline ETL, puis la partie analyse et visualisation.

Méthode

Le développement a été progressif, avec des validations à chaque étape. Chaque commit correspond à une fonctionnalité ou une correction identifiable.

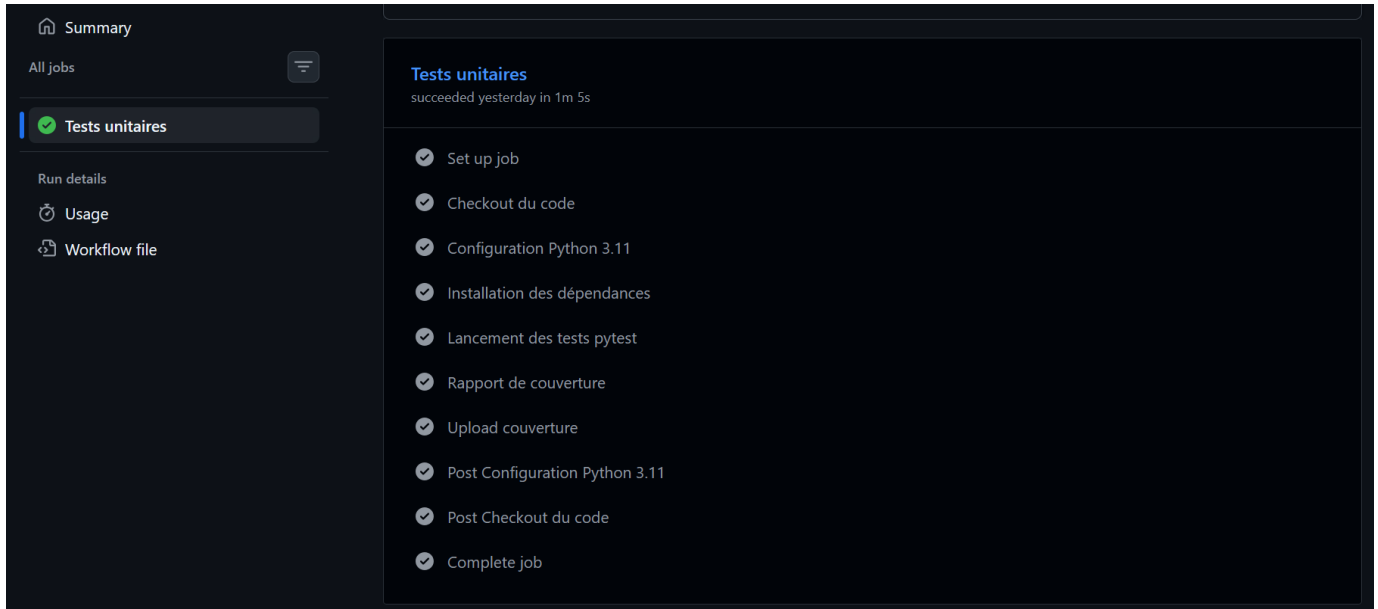
Dépôt GitHub

Le projet est disponible sur GitHub et structuré de manière claire. On y retrouve les modules ETL, le dashboard, les notebooks, l’infrastructure Docker, les tests et les scripts.

Tests et CI/CD

Le projet contient 37 tests unitaires couvrant l'ensemble du pipeline. Tous les tests passent correctement.

Une intégration continue via GitHub Actions assure la validation automatique à chaque modification sur la branche principale.



GitHub Actions avec pipeline CI/CD valide.

```
(.venv-1) PS C:\Users\samir\Desktop\data-pipeline-emploi> pytest tests/ -v
tests/test_cleaner.py::TestDataCleaner::test_nettoyer_scrutins_prive PASSED [ 37%]
tests/test_loaders.py::TestMinioLoader::test_instanciacion PASSED [ 40%]
tests/test_loaders.py::TestMinioLoader::test_attributs_connexion PASSED [ 43%]
tests/test_loaders.py::TestMinioLoader::test_upload_appelle_client PASSED [ 45%]
tests/test_loaders.py::TestMinioLoader::test_bucket_cree_si_absent PASSED [ 48%]
tests/test_loaders.py::TestPostgresLoader::test_instanciacion PASSED [ 51%]
tests/test_loaders.py::TestPostgresLoader::test_attributs_connexion PASSED [ 54%]
tests/test_loaders.py::TestPostgresLoader::test_charger_deputes PASSED [ 56%]
tests/test_loaders.py::TestPostgresLoader::test_charger_scrutins PASSED [ 59%]
tests/test_loaders.py::TestIntegrationPipeline::test_pipeline_import PASSED [ 62%]
tests/test_loaders.py::TestIntegrationPipeline::test_scraper_import PASSED [ 64%]
tests/test_loaders.py::TestIntegrationPipeline::test_cleaner_import PASSED [ 67%]
tests/test_loaders.py::TestIntegrationPipeline::test_loaders_import PASSED [ 70%]
tests/test_loaders.py::TestIntegrationPipeline::test_dataclasse_depute_attributs PASSED [ 72%]
tests/test_loaders.py::TestIntegrationPipeline::test_dataclasse_scrutin_attributs PASSED [ 75%]
tests/test_scraper.py::TestDeputesCollector::test_instanciacion PASSED [ 78%]
tests/test_scraper.py::TestDeputesCollector::test_parse_depute_valide PASSED [ 81%]
tests/test_scraper.py::TestDeputesCollector::test_parse_depute_nil_xml PASSED [ 83%]
tests/test_scraper.py::TestDeputesCollector::test_parse_depute_champs_manquants PASSED [ 86%]
tests/test_scraper.py::TestDeputesCollector::test_parse_depute_attributs_complets PASSED [ 89%]
tests/test_scraper.py::TestScrutinsCollector::test_instanciacion PASSED [ 91%]
tests/test_scraper.py::TestScrutinsCollector::test_parse_scrutin_valide PASSED [ 94%]
tests/test_scraper.py::TestScrutinsCollector::test_parse_scrutin_sort_rejete PASSED [ 97%]
tests/test_scraper.py::TestScrutinsCollector::test_parse_scrutin_valeurs_numeriques PASSED [100%]

===== 37 passed in 2.12s =====
(.venv-1) PS C:\Users\samir\Desktop\data-pipeline-emploi> █
```

Résultat de pytest montrant 37 tests réussis.

8) Conclusion et perspectives

Bilan

Les objectifs du projet sont atteints. Le pipeline fonctionne de bout en bout, l'infrastructure est stable, les données sont structurées et exploitables, et le dashboard permet une exploration simple et efficace.

Sur le plan technique, le projet couvre l'ensemble de la chaîne data moderne, de la collecte jusqu'à la visualisation.

Limites

Certaines limites restent présentes, notamment l'absence des votes nominatifs et le manque de déploiement continu du dashboard.

Perspectives

Plusieurs améliorations sont possibles.

Ajouter les votes individuels permettrait d'analyser les coalitions et la cohésion des groupes politiques.

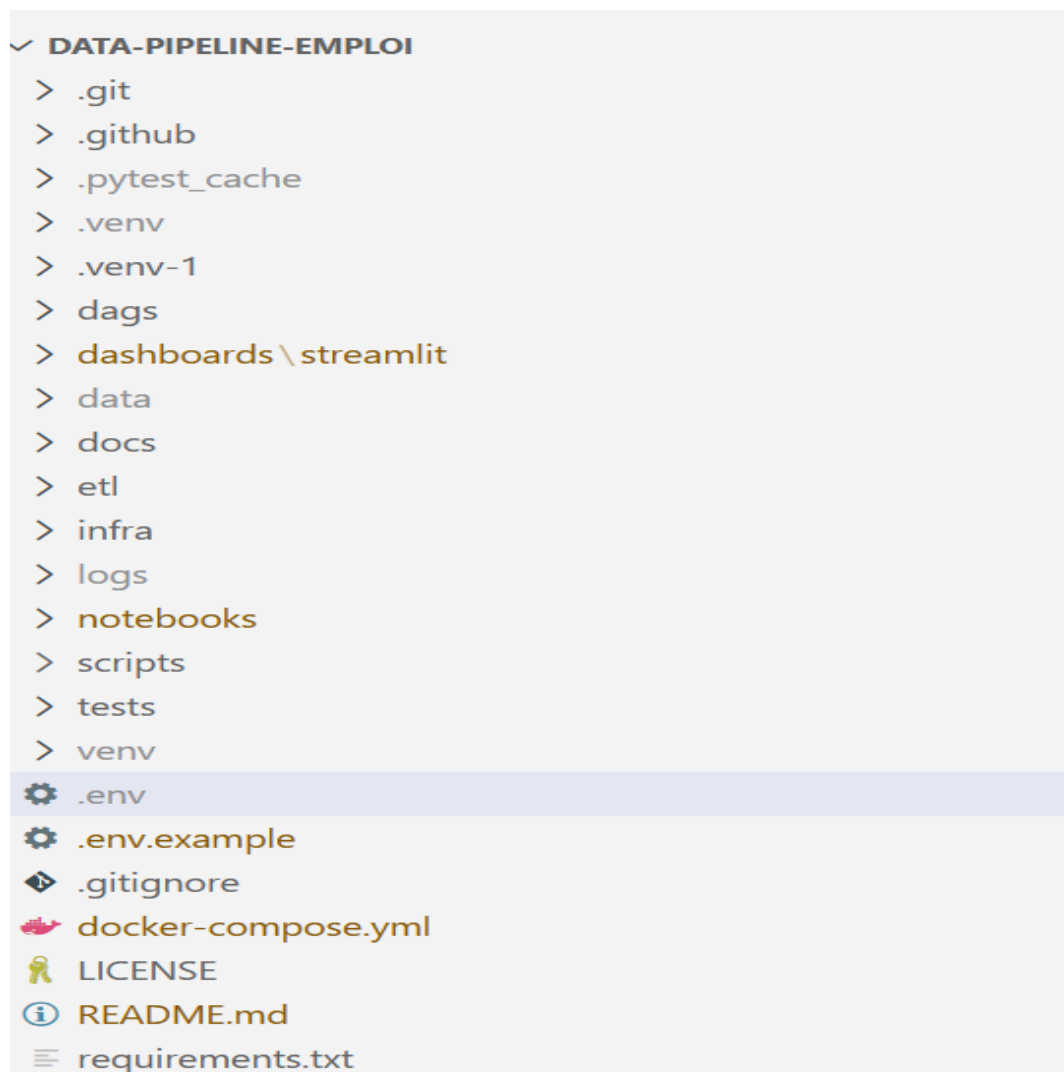
Déployer le dashboard en continu rendrait le projet accessible en permanence.

Enfin, élargir les sources de données parlementaires permettrait d'enrichir davantage les analyses produites.

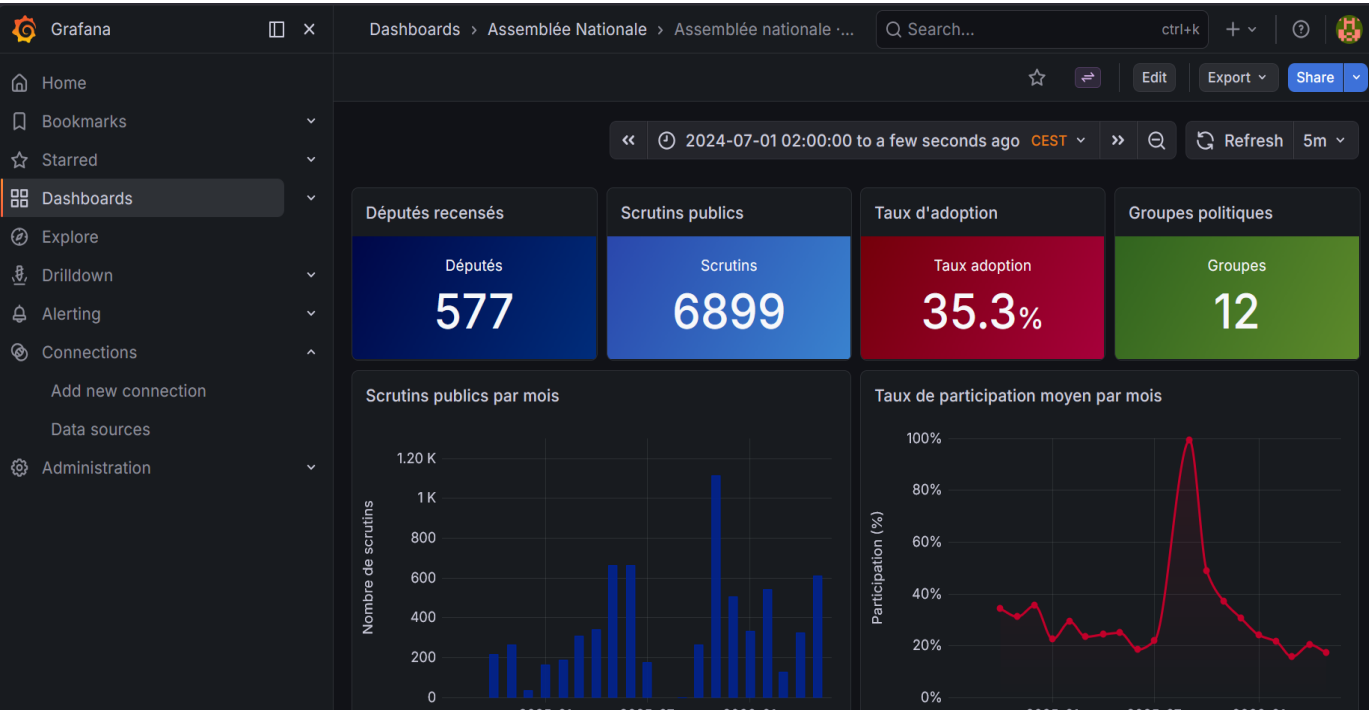
Annexes

Stack technique

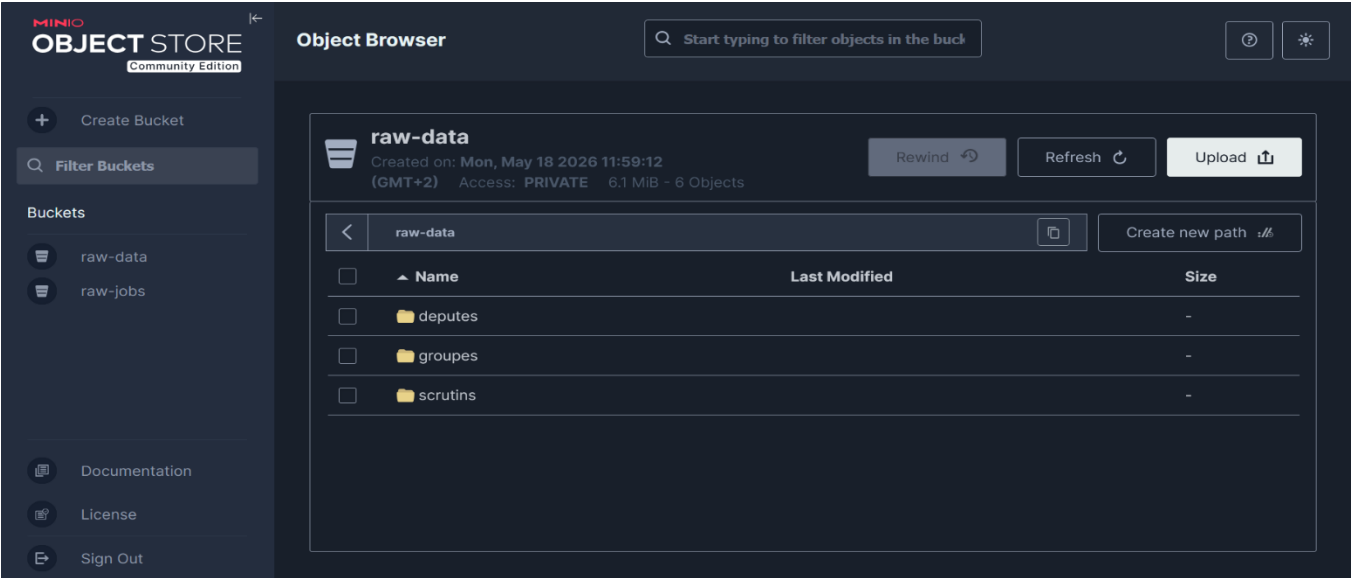
Python 3.13, requests, BeautifulSoup, pandas, numpy, scipy, matplotlib, seaborn, plotly, SQLAlchemy, psycpg2, minio, boto3, loguru, python-dotenv, streamlit, pytest, pytest-cov, Apache Airflow 2.9, PostgreSQL 15, MinIO, Prometheus, Grafana, Docker, Docker Compose, GitHub Actions, environnement de test AWS EC2.



Arborescence du dépôt Github.



Dashboard Grafana de monitoring.



Interface MinIO

Welcome X jobs_db/jobs_user@postgres_jobs* X

jobs_db/jobs_user@postgres_jobs

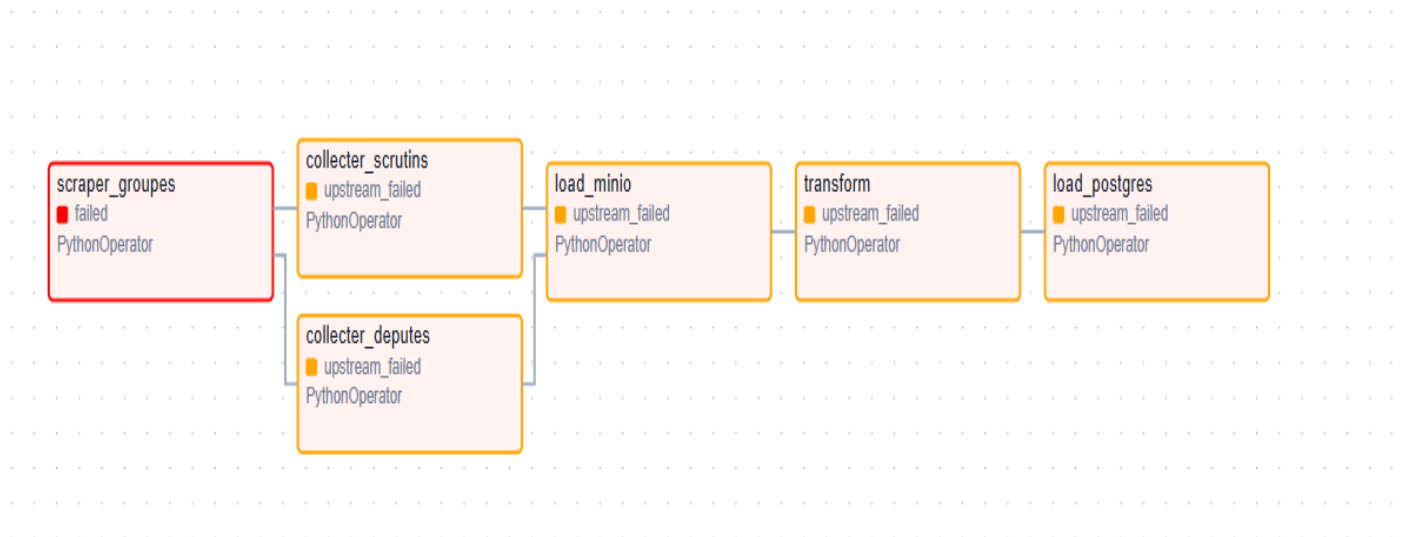
No limit

Data Output Messages Notifications

Showing rows: 1 to 5 Page No: 1

	table_name	
	name	
1	job_offers	
2	groupes_politiqu...	
3	deputes	
4	scrutins	
5	pipeline_stats	

Interface pgAdmin avec les tables PostgreSQL.



Interface Airflow avec le DAG.